

Final Lab Task Submission

Artificial Intelligence — Final Lab Task

Name: Minhazul Abedin

ID: _____

Section: _____

Semester: Summer 25-26

Task 1 — Goal-Based Agent: Automated Water Purification System

A **goal-based agent** reads the sensor state of incoming water (Turbidity in NTU, TDS in ppm, Bacterial contamination, pH) and repeatedly applies **condition-action rules** until every safety threshold is met: Turbidity ≤ 1 NTU, TDS ≤ 300 ppm, Bacteria = False and pH between 6.5 and 8.5. Available actions are Sedimentation, Filtration, Reverse Osmosis, UV Treatment and Chemical Addition. The updated state is printed after every action.

Code Screenshot

```
task1_water_purification_agent.py

1 # =====
2 # Final Lab Task 1 - Goal-Based Agent
3 # Automated Water Purification System
4 # =====
5
6 # ----- GOAL STATE (safety thresholds) -----
7 MAX_TURBIDITY = 1.0      # NTU
8 MAX_TDS       = 300      # ppm
9 SAFE_PH_LOW   = 6.5
10 SAFE_PH_HIGH  = 8.5
11 TARGET_PH    = 7.2      # set-point used by the chemical doser
12
13
14 def show(state):
15     """Return the water state as a readable string."""
16     return ("Turbidity = %.1f, TDS = %.0f, Bacteria = %s, pH = %.1f"
17             % (state["Turbidity"], state["TDS"], state["Bacteria"], state["pH"]))
18
19
20 def goal_test(state):
21     """Return True only when every safety threshold is satisfied."""
22     return (state["Turbidity"] <= MAX_TURBIDITY and
23             state["TDS"] <= MAX_TDS and
24             state["Bacteria"] is False and
25             SAFE_PH_LOW <= state["pH"] <= SAFE_PH_HIGH)
26
27
28 def decide_action(state):
29     """
30     Condition-Action rules.
31     Returns (reason, action_name) for the first unsatisfied goal condition,
32     or None when the goal state has already been reached.
33     """
34     if state["Turbidity"] > 3.0:
35         return ("Turbidity > 1", "Sedimentation")
36     if state["Turbidity"] > MAX_TURBIDITY:
37         return ("Turbidity > 1", "Filtration")
38     if state["TDS"] > MAX_TDS:
39         return ("TDS > 300", "Reverse Osmosis")
40     if state["Bacteria"] is True:
41         return ("Bacteria = True", "UV Treatment")
42     if state["pH"] < SAFE_PH_LOW:
43         return ("pH < 6.5", "Add Chemicals")
44     if state["pH"] > SAFE_PH_HIGH:
45         return ("pH > 8.5", "Add Chemicals")
46     return None
```

```

task1_water_purification_agent.py (continued)
49 def execute(action, state):
50     """Apply the effect of the chosen purification action on the water."""
51     if action == "Sedimentation":           # heavy particles settle
52         state["Turbidity"] = round(state["Turbidity"] * 0.10, 2)
53     elif action == "Filtration":            # fine particle removal
54         state["Turbidity"] = round(state["Turbidity"] * 0.40, 2)
55     elif action == "Reverse Osmosis":       # dissolved solids removal
56         state["TDS"] = round(state["TDS"] * 0.40, 2)
57     elif action == "UV Treatment":          # bacteria elimination
58         state["Bacteria"] = False
59     elif action == "Add Chemicals":         # dose towards neutral pH
60         state["pH"] = TARGET_PH
61     return state
62
63
64 def water_purification_agent(state):
65     """Goal-based agent: keep acting until the water reaches the goal state."""
66     print("Initial water quality:", show(state))
67     print("-" * 62)
68
69     while not goal_test(state):
70         rule = decide_action(state)
71         if rule is None:
72             break
73         reason, action = rule
74         print("→ %s → Action Taken: %s" % (reason, action))
75         state = execute(action, state)
76         print("    Updated state: " + show(state))
77
78     print("-" * 62)
79     print("Water quality after treatment:", show(state))
80     print("Goal Achieved: Water is purified and safe for drinking.")
81     return state
82
83
84 # ----- MAIN -----
85 if __name__ == "__main__":
86     water_sample = {
87         "Turbidity": 5.0,
88         "TDS": 700,
89         "Bacteria": True,
90         "pH": 5.8
91     }
92     water_purification_agent(water_sample)

```

Output Screenshot

```

Terminal — Output

$ python3 task1_water_purification_agent.py
Initial water quality: Turbidity = 5.0, TDS = 700, Bacteria = True, pH = 5.8
-----
→ Turbidity > 1 → Action Taken: Sedimentation
   Updated state: Turbidity = 0.5, TDS = 700, Bacteria = True, pH = 5.8
→ TDS > 300 → Action Taken: Reverse Osmosis
   Updated state: Turbidity = 0.5, TDS = 280, Bacteria = True, pH = 5.8
→ Bacteria = True → Action Taken: UV Treatment
   Updated state: Turbidity = 0.5, TDS = 280, Bacteria = False, pH = 5.8
→ pH < 6.5 → Action Taken: Add Chemicals
   Updated state: Turbidity = 0.5, TDS = 280, Bacteria = False, pH = 7.2
-----
Water quality after treatment: Turbidity = 0.5, TDS = 280, Bacteria = False, pH = 7.2
Goal Achieved: Water is purified and safe for drinking.

```

Task 2 — Simple Rule-Based Vacuum Cleaning Agent

A **simple reflex (rule-based) agent** works in a 4×4 grid world where every cell is either Dirty (**D**) or Clean (**.**) and the agent is drawn as **A**. At each step the agent **perceives** whether its current cell is dirty, **decides** its action (clean, move right, or move down to the next row) and **executes** it, printing the environment after every action until the whole grid is clean.

Code Screenshot

```
task2_vacuum_cleaning_agent.py

1  # =====
2  # Final Lab Task 2 - Simple Rule-Based Vacuum Cleaning Agent
3  # 2-D grid world: 'D' = Dirty cell, '.' = Clean cell, 'A' = Agent
4  # =====
5
6  ROWS = 4
7  COLS = 4
8
9  # The environment: each cell is either Dirty ('D') or Clean ('.')
10 environment = [
11     ['D', '.', 'D', '.'],
12     ['D', '.', 'D', '.'],
13     ['D', '.', '.', '.'],
14     ['D', 'D', 'D', 'D']
15 ]
16
17
18 def display(grid, agent_row, agent_col):
19     """Print the current world. The agent's own cell is shown as 'A'."""
20     for r in range(ROWS):
21         for c in range(COLS):
22             if r == agent_row and c == agent_col:
23                 print('A', end=' ')
24             else:
25                 print(grid[r][c], end=' ')
26         print()
27     print('-' * 20)
28
29
30 def perceive(grid, row, col):
31     """PERCEPTION: is the agent's current location dirty?"""
32     return grid[row][col] == 'D'
33
34
35 def clean(grid, row, col):
36     """ACTION: suck the dirt in the current cell."""
37     grid[row][col] = '.'
38
39
40 def next_position(row, col):
41     """
42     ACTION: decide the next move.
43     - move RIGHT while cells remain in the current row
44     - otherwise move DOWN to the start of the next row
45     - return None when the whole grid has been scanned
46     """
47     if col < COLS - 1:
48         return row, col + 1
49     elif row < ROWS - 1:
50         return row + 1, 0
51     return None
```

task2_vacuum_cleaning_agent.py (continued)

```
53 def vacuum_agent(grid):
54     """Simple rule-based agent: perceive -> decide -> act, over the whole grid."""
55     row, col = 0, 0
56     cleaned = 0
57
58     print("Initial Environment:")
59     display(grid, row, col)
60
61     while True:
62         # RULE 1: if the current cell is dirty, then clean it
63         if perceive(grid, row, col):
64             clean(grid, row, col)
65             cleaned += 1
66             print("Cleaned cell at (%d, %d)" % (row, col))
67             display(grid, row, col)
68
69         # RULE 2: otherwise move to the next cell of the environment
70         step = next_position(row, col)
71         if step is None:
72             break
73         row, col = step
74         display(grid, row, col)
75
76     display(grid, row, col)
77     print("Cleaning complete. Total cells cleaned:", cleaned)
78
79
80 # ----- MAIN -----
81 if __name__ == "__main__":
82     vacuum_agent(environment)
83
```

Output Screenshot

Full console output, shown in two columns to fit one page — read the left column top-to-bottom first, then the right column.

Terminal — Output

```
$ python3 task2_vacuum_cleaning_agent.py
```

Initial Environment:

```
A . D .  
D . D .  
D . . .  
D D D D
```

Cleaned cell at (0, 0)

```
A . D .  
D . D .  
D . . .  
D D D D
```

```
. A D .  
D . D .  
D . . .  
D D D D
```

```
. . A .  
D . D .  
D . . .  
D D D D
```

Cleaned cell at (0, 2)

```
. . A .  
D . D .  
D . . .  
D D D D
```

```
. . . A  
D . D .  
D . . .  
D D D D
```

```
. . . .  
A . D .  
D . . .  
D D D D
```

Cleaned cell at (1, 0)

```
. . . .  
A . D .  
D . . .  
D D D D
```

```
. . . .  
. A D .  
D . . .  
D D D D
```

```
. . . .  
. . A .  
D . . .  
D D D D
```

Cleaned cell at (1, 2)

```
. . . .  
. . A .  
D . . .  
D D D D
```

```
. . . .  
. . . A  
D . . .  
D D D D
```

```
. . . .  
. . . .  
A . . .  
D D D D
```

Cleaned cell at (2, 0)

```
. . . .  
. . . .  
A . . .  
D D D D
```

```
. . . .  
. . . .  
. A . .  
D D D D
```

```
. . . .  
. . . .  
. . A .  
D D D D
```

```
. . . .  
. . . .  
. . . A  
D D D D
```

```
. . . .  
. . . .  
. . . .  
A D D D
```

Cleaned cell at (3, 0)

```
. . . .  
. . . .  
. . . .  
A D D D
```

```
. . . .  
. . . .  
. . . .  
. A D D
```

Cleaned cell at (3, 1)

```
. . . .  
. . . .  
. . . .  
. A D D
```

```
. . . .  
. . . .  
. . . .  
. . A D
```

Cleaned cell at (3, 2)

```
. . . .  
. . . .  
. . . .  
. . A D
```

```
. . . .  
. . . .  
. . . .  
. . . A
```

Cleaned cell at (3, 3)

```
. . . .  
. . . .  
. . . .  
. . . A
```

```
. . . .  
. . . .  
. . . .  
. . . A
```

Cleaning complete. Total cells cleaned: 9